NLU course project - Part 1A and Part 1B

Emanuele Ricca (258437)

University of Trento

emanuele.ricca@studenti.unitn.it

1. Introduction

This report explores the development, optimization, and evaluation of autoregressive language models on the Penn TreeBank corpus [3]. The study is structured across two major modeling paradigms: architectural scaling from scratch and parameter-efficient domain adaptation.

We first investigate the incremental optimization of a custom, scratch-built GPT2-style model [5]. Starting from a highly constrained configuration, network capacity is systematically scaled up by tuning hidden dimensions, multi-head attention mechanisms, layer depth, and feed-forward network capacities. To combat overfitting in these expanded parameter spaces, four independent dropout layers [6] and embedding weight-tying techniques [4] are rigorously isolated and evaluated against a mandatory validation threshold (PPL < 250).

We then contrast this resource-intensive setup against Parameter-Efficient Fine-Tuning (PEFT) by engineering a manual implementation of Low-Rank Adaptation (LoRA) [1] on a pre-trained GPT2 backbone [5]. Trainable low-rank adapter bottlenecks are integrated directly into the attention projection matrices while the core network remains entirely frozen. The objective is to maximize parameter efficiency and surpass the performance profiles of the scratch-optimized variants under identical perplexity criteria. Generative AI tools were utilized strictly to assist with scripting logic verification, debugging, and document formatting.

2. Implementation Details

The experimental framework coordinates an entirely custom autoregressive pipeline alongside a low-rank fine-tuning adapter interface.

The custom scratch-built architecture consists of token and learnable position embeddings, masked causal self-attention pathways [7], and a linear language modeling head. The network footprint was systematically scaled up from a minimal baseline ($d_{\text{model}} = 20, n_{\text{heads}} = 1, num_{\text{layers}} = 1, ff_{\text{dim}} = 20$) to an expanded configuration ($d_{\text{model}} = 64, n_{\text{heads}} = 2, num_{\text{layers}} = 2, ff_{\text{dim}} = 128$). Regularization incorporates four distinct dropout modules [6]: embedding, attention, projection, and feed-forward pathways. Weight-tying mechanics [4] are integrated to bind the parameters of the input embedding matrix directly with the final linear output layer. Optimization is driven by AdamW [2] over 20 epochs with a 3-epoch linear warmup, an early-stopping patience of 3, and a learning rate adjusted from 0.01 down to 0.005 to stabilize the expanded configuration sizes.

For parameter-efficient fine-tuning, LoRA adapters [1] were manually constructed from scratch without third-party PEFT libraries. Low-rank decomposition pathways are mapped to alter the linear query, key, and value projection transformations within each attention block [7]. For any frozen base weight matrix $W_0 \in \mathbb{R}^{d \times k}$, the adapted forward pass is computed as:

$$W = W_0 + \Delta W = W_0 + \frac{\alpha}{r} BA$$

where $A \in \mathbb{R}^{r \times k}$ and $B \in \mathbb{R}^{d \times r}$ are the trainable low-rank adapters, $r = 4$ represents the rank bottleneck, and $\alpha \in \{8, 16\}$ is the constant scaling multiplier. Core base parameters are strictly frozen. Fine-tuning uses the AdamW optimizer [2], a tuned learning rate of 0.0005, and a 3-epoch linear warmup.

3. Results and Evaluation

Performance is evaluated via cross-entropy loss translated into Perplexity (PPL $= \exp(\text{loss})$). Table 1 logs the metrics gathered across the design phases of the custom scratch model, while Table 2 breaks down the performance profiles of the manual LoRA runs.

Table 1: *Incremental optimization perplexities for Part 1.A (From Scratch).*

Incremental Modification Variant	Dev PPL	Test PPL
Baseline Layout ($lr = 0.01$)	56.68	49.51
+ Bigger d_{model} (32, $ff_{\text{dim}} = 64$)	46.68	41.51
+ More Layers ($num_{\text{layers}} = 2$)	46.75	41.45
+ More Heads ($n_{\text{heads}} = 2$)	46.18	40.77
+ Embedding Dropout (0.2)	48.65	43.02
+ Embedding Dropout (0.05)	46.52	41.25
+ Attention Dropout (0.05)*	46.56	N/A
+ Feed-Forward Dropout (0.1)	46.17	40.96
+ Projection Dropout (0.05)	46.91	41.57
+ Weight Tying (WT, $lr = 0.01$)	54.45	48.10
+ WT + FF Drop 0.1 ($lr = 0.005$)	50.53	44.77
+ WT + FF Drop 0.1 (Scaled Up)	**43.47**	**38.81**

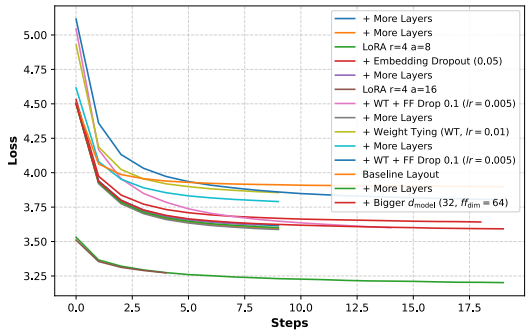
Table 2: *LoRA fine-tuning parameters and perplexity metrics for Part 1.B.*

Configuration Profile	Rank	Alpha	Dev PPL	Test PPL
LoRA Run 1*	4	16	24.29	N/A
LoRA Run 2 (Best)	**4**	**8**	**23.24**	**20.99**

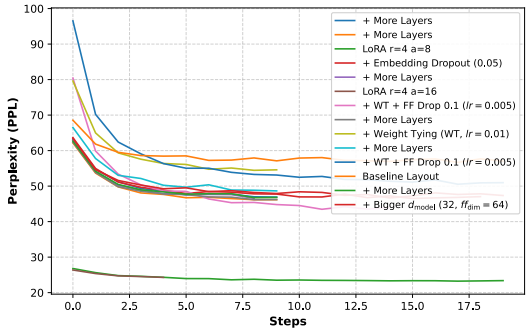
Empirical metrics indicate highly divergent optimization curves between the two approaches. For the models trained from scratch, expanding d_{model} from 20 to 32 delivered the most significant drop in perplexity. Due to the compact nature of the corpus, severe overfitting was absent, rendering heavy embedding and projection dropouts [6] counterproductive; only a mild feed-forward dropout at 0.1 provided structural regularization. Weight tying [4] initially harmed perplexity due to representational bottlenecks on an narrow parameter budget, but its reduction in parameter volume enabled scaling the network dimensions safely up to $d_{\text{model}} = 64$ and $ff_{\text{dim}} = 128$, resulting in a peak test PPL of 38.81.

Conversely, the manual LoRA framework completely bypasses these initialization hurdles by adapting a powerful pre-trained representation base [5]. This strategy securely met all performance objectives, achieving a vastly superior test perplexity of 20.99 [1].

To visually analyze these structural differences, Figure **??** presents the synchronized TensorBoard trajectories comparing the training and validation characteristics of both implementations.



(a) *Unified Training Loss Track*



(b) *Development Perplexity Profile*

As mapped in Figures 1a and 1b, the models built from scratch demonstrate a classical steep optimization curve. They begin with very high training loss and validation perplexity metrics, requiring a longer sequence of epochs to map structural English relationships. In comparison, the LoRA runs demonstrate an exceptionally flat convergence profile. Starting instantly at Epoch 1 with a development perplexity near 26, the adapters converge and plateau within 3 to 4 epochs due to the foundational features embedded in the pre-trained backbone.

Figure **??** underscores that while individual steps of the low-rank adapter are wall-clock slower (resulting from back-propagating gradients through the large frozen backbone), the total computational budget is heavily reduced since the network completely avoids the prolonged warming and learning cycles required when training from scratch. The configuration with $\alpha = 16$ was aborted early as its dev tracking proved less stable than the balanced $\alpha = 8$ run.

4. References

[1] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. Lora: Low-rank adaptation of large language models. *ICLR*, 2022.

[2] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. *arXiv preprint arXiv:1711.05101*, 2017.

[3] Mitchell P Marcus, Beatrice Santorini, and Mary Ann Marcinkiewicz. Building a large annotated corpus of english: The penn treebank. *Computational linguistics*, 19(2):313–330, 1993.

[4] Ofir Press and Lior Wolf. Using the output embedding to improve language models. *arXiv preprint arXiv:1608.05859*, 2016.

[5] Alec Radford, Jeffrey Wu, Rewon Child, Dario Luan, Dario Amodei, and Ilya Sutskever. Language models are unsupervised multitask learners. 2019.

[6] Nitish Srivastava, Geoffrey Hinton, Alex Krizhevsky, Ilya Sutskever, and Ruslan Salakhutdinov. Dropout: a simple way to prevent neural networks from overfitting. *The journal of machine learning research*, 15(1):1929–1958, 2014.

[7] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Luan Kaiser, and Illia Polosukhin. Attention is all you need. *NeurIPS*, 2017.